TECHNOLOGIE INTERNETOWE · LABORATORIUM 1

HTML, klasy i CSS + GitHub

Przypomnienie podstaw i start projektu semestralnego

mgr inż. Jan Cybulski · Creospace · UKW Bydgoszcz

DZISIAJ · OK. 2 GODZINY

Plan zajęć

Czas	Blok
0–15 min	Zasady zaliczenia, projekt semestralny, brief klienta
15–40 min	GitHub: konto, repozytorium, pierwszy commit i push
40–60 min	Przypomnienie HTML: struktura i znaczniki semantyczne
60–85 min	Klasy, id, podpięcie CSS, selektory, specyficzność
85–115 min	Zadanie: strona dla klienta + GitHub Pages
115–120 min	Ostatni push i link do repozytorium

ZALICZENIE

Jak zdobyć ocenę

Repozytorium

Każde laby kończą się git push. Oceniam to, co jest w repo.

Sprawdziany

Krótkie, na zajęciach: test albo działający etap projektu.

Kolokwium

Działający projekt plus pytania pisemne.

Projekt semestralny: strona dla klienta, którą na kolejnych labach rozbudujecie do prostego CMS-a.

Bez repozytorium nie ma oceny.

Każde zajęcia kończą się poleceniem git push.

GITHUB · OBOWIĄZKOWO

Repozytorium w 4 krokach

1
Załóż konto na github.com

2
Publiczne repo ti-lab-nazwisko z README

3
Konfiguracja Gita, jednorazowo

4
Clone i praca w katalogu repo

```
3 › git config --global user.name "Imię Nazwisko"
3 › git config --global user.email "twoj@email.pl"
4 › git clone https://github.com/TWOJ-LOGIN/ti-lab-nazwisko.git
4 › cd ti-lab-nazwisko
```

PO KAŻDEJ WIĘKSZEJ ZMIANIE

Trzy polecenia na co dzień

KROK 1

`git add .`

Wybierz zmiany do zapisania

KROK 2

`git commit`

Zapisz je lokalnie w historii

KROK 3

`git push`

Wyślij historię na GitHuba

Logowanie: GitHub nie przyjmuje hasła w terminalu. Zaloguj się przez VS Code („Sign in with GitHub”) albo w oknie przeglądarki, które otworzy Git.

ZASADY REPOZYTORIUM

Commity jak w software house’ie

Dobrze	Źle
`lab1: struktura strony głównej`	`zmiany`
`lab1: nawigacja we flexie`	`asdf`
`lab1: cennik jako tabela`	`final_final2`

- Format opisu: labN: co zrobiono
- Kilka małych commitów zamiast jednego wielkiego „final”
- Opis commita czyta ktoś inny: kolega z zespołu, klient, prowadzący

PRZYPOMNIENIE HTML

Szkielet każdej strony

```
<!DOCTYPE html>
<html lang="pl">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Szprycha – serwis rowerowy</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">…</header>
  <main>…</main>
  <footer id="kontakt">…</footer>
</body>
</html>
```

`lang="pl"`

Czytnik ekranu wie, jak czytać tekst

`viewport`

Strona dobrze działa na telefonie

`link`

CSS w osobnym pliku, nie w HTML

SEMANTYKA

Każdy element mówi, czym jest



Znaczniki semantyczne zamiast samych div-ów. Czytnik ekranu i Google rozumieją układ strony.

Jeden h1 na stronę, nagłówki bez przeskakiwania poziomów.

Każdy obrazek ma alt z opisem, co przedstawia.

To jest WCAG 2.1 i SEO w praktyce.

KLASY I ID

Jak HTML łączy się z CSS

W HTML	W CSS	Kiedy używać
class="card"	.card	Styl wielokrotnego użytku; element może mieć kilka klas
id="kontakt"	#kontakt	Element unikalny: kotwica w linku, JavaScript
sam znacznik	nav a	Style bazowe, np. wszystkie linki w menu

Klasa = rodzaj elementu. Id = ten jeden konkretny element.

CSS W OSOBNYM PLIKU

Klasy w praktyce

```
/* style.css */
.menu { display: flex; gap: 16px; }
.menu a { color: #4426FF; text-decoration: none; }
.card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; }
.card--highlight { border-color: #4426FF; background: #EEF; }
<!-- index.html -->
<article class="card card--highlight">…</article>
```

DOBRA NAZWA KLASY

`.card .menu .price-table`

Mówi, czym element jest

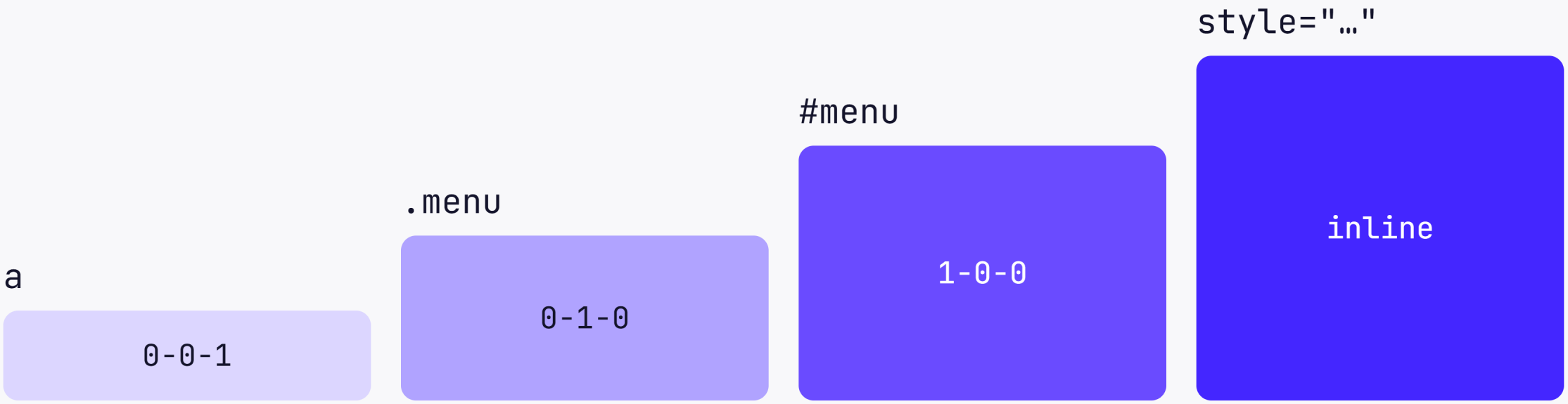
ZŁA NAZWA KLASY

`.niebieski .duzy .lewo`

Mówi tylko, jak wygląda

SPECYFICZNOŚĆ

Która reguła wygrywa?



Przy równej sile wygrywa reguła zapisana niżej w pliku. Unikaj !important.

ĆWICZENIE NA ŻYWO

Kto wygra?

```
<p id="cena" class="price">99 zł</p>

#cena  { color: green; }
.price { color: red; }
p      { color: blue; }
```

Jakiego koloru będzie cena?

A. niebieski

B. czerwony

C. zielony

C. Zielony. Id wygrywa, choć jest zapisane najwyżej.

ZADANIE NA DZIŚ · BRIEF OD KLIENTA

„Prowadzę małą pracownię rowerową Szprycha. Potrzebuję prostej strony z ofertą, cennikiem i kontaktem. Ma się dobrze czytać na telefonie. Nie mam logo, wystarczy nazwa.”

Pracownia Rowerowa Szprycha, Bydgoszcz · klient fikcyjny

BRIEF PRZEŁOŻONY NA WYMAGANIA

Co musi mieć strona

1. index.html + style.css, CSS tylko w pliku
2. header, nav, main, 2× section, footer
3. Menu we flexie, linki do #sekcji
4. Min. 3 karty .card, jedna card--highlight
5. Cennik jako tabela z nagłówkami th
6. alt przy obrazkach, lang="pl", viewport
7. Min. 3 commity z opisem lab1: …
8. Strona działa na GitHub Pages

PRACA WŁASNA · OK. 30 MINUT

4 zadania, 4 commity

1

Szkielet strony

```
lab1: szkielet strony
```

2

Nawigacja

```
lab1: nawigacja
we flexie
```

3

Karty usług

```
lab1: karty uslug
```

4

Cennik i kontakt

```
lab1: cennik i kontakt
```

Po każdym zadaniu commit. Nie czekaj do końca zajęć.

Utknąłeś na 5 minut? Podnieś rękę. Kod startowy jest w materiale HTML.

PRACA WŁASNA · ZADANIE 1 Z 4 · OK. 7 MIN

Szkielet strony

- Utwórz index.html i style.css w katalogu repozytorium
- Szkielet z lang="pl", viewport i link do style.css
- header z nazwą i menu (kod obok)
- main z sekcjami #uslugi i #cennik, footer z id="kontakt"

```
<header class="site-header">
  <p class="logo">Szprycha</p>
  <nav class="menu">
    <a href="#uslugi">Usługi</a>
    <a href="#cennik">Cennik</a>
    <a href="#kontakt">Kontakt</a>
  </nav>
</header>
```

GOTOWE, GDY

Strona otwiera się w przeglądarce, a linki w menu przewijają do sekcji.

```
git commit -m "lab1: szkielet strony"
```

PRACA WŁASNA · ZADANIE 2 Z 4 · OK. 7 MIN

Nawigacja we flexie

- Nazwa po lewej, menu po prawej, w jednej linii
- Uzupełnij luki ___ w kodzie obok
- Linki bez podkreślenia, inny kolor po najechaniu
- W DevTools (F12) wyłącz każdą właściwość i zobacz, co robi

```
.site-header {
  display: ___;
  justify-content: ___;
  align-items: center;
}
.menu { display: flex; gap: ___; }
.menu a { text-decoration: none; }
.menu a:hover { color: ___; }
```

GOTOWE, GDY

Nazwa i menu są w jednym rzędzie, a link po najechaniu zmienia kolor.

```
git commit -m "lab1: nawigacja
we flexie"
```

PRACA WŁASNA · ZADANIE 3 Z 4 · OK. 8 MIN

Karty usług

- Min. 3 karty article.card: nazwa, opis, cena
- Jedna karta z card--highlight i napisem „Najczęściej wybierane”
- Karty obok siebie, równej szerokości
- Na wąskim oknie same schodzą pod siebie

```
<div class="cards">
  <article class="card">
    <h3>Przegląd</h3>
    <p>Hamulce i przerzutki.</p>
    <p class="card__price">od 120 zł</p>
  </article>
</div>
.cards { display: flex; flex-wrap: wrap; gap: 24px; }
.card { flex: 1 1 260px; }
```

GOTOWE, GDY

Na laptopie 3 karty w rzędzie, po zwężeniu okna jedna pod drugą.

```
git commit -m "lab1: karty uslug"
```

PRACA WŁASNA · ZADANIE 4 Z 4 · OK. 8 MIN

Cennik i kontakt

- Tabela z thead i th, min. 4 usługi
- Ceny wyrównane do prawej: .num { text-align: right; }
- Stopka: adres, telefon (tel:) i e-mail (mailto:)

```
<table class="price-table">
  <thead>
    <tr><th>Usługa</th><th class="num">Cena</th></tr>
  </thead>
  <tbody>
    <tr><td>Przegląd</td><td class="num">120 zł</td></tr>
  </tbody>
</table>
<a href="tel:+48500100200">500 100 200</a>
```

GOTOWE, GDY

Ceny stoją równo w kolumnie, a telefon dzwoni po kliknięciu na komórce.

```
git commit -m "lab1: cennik i kontakt"
```

DLA SZYBKICH I DO DOMU

Bonus i praca domowa

BONUS · NA PLUS

- Poniżej 600 px menu w kolumnie (media query)
- Karta po najechaniu unosi się o 6 px i dostaje cień
- Lighthouse w Chrome: Dostępność min. 90 pkt

```
@media (max-width: 600px) {
  .site-header { flex-direction: column; }
}
```

PRACA DOMOWA

Dokończ zadania 1–4 i zrób git push przed następnymi zajęciami.

Strona ma działać pod linkiem GitHub Pages.

PUBLIKACJA

Strona online w minutę: GitHub Pages

Settings → Pages → Deploy from a branch → main / (root)

PO 1–2 MINUTACH STRONA JEST TUTAJ

```
https://TWOJ-LOGIN.github.io/ti-lab-nazwisko/
```

Na koniec wyślij prowadzącemu dwa linki: do repozytorium i do działającej strony.

Koniec?
Najpierw git push.

Następne laby: JavaScript, XML i SVG. Dokładamy do strony Szprychy dynamikę i grafikę.

mgr inż. Jan Cybulski · Creospace · creospace.eu